

GRIT — Master Brief

Last updated: 2026-06-14 22:17 PDT · GRIT (GPU Reuse under Interleaved Traffic) · the single document to learn agents, the serving ecosystem, the literature, the tools, the project, and to prep for the NVIDIA inference-PM role ·

Target: MLSys 2027 Industry Track (~Oct 2026, provisional) · public benchmarks + open infrastructure only

How to read this document

This is the **master educational brief** for the project. It is written so a newcomer can learn the whole area from scratch: technical terms are explained in parentheses the first time they appear and collected in the

Key terms box. It moves from **how inference works** → **what agents are** → **the ecosystem** → **the literature** (every verified paper, summarized) → **the tools** → **the GRIT project** → **open questions**.

Citations marked ✓ are verified against the live ledger (02-literature/sota-verified-2026.md), which is the source of truth; anything unverified is quarantined in the box at the very end. **This brief is kept in sync with the repo and re-timestamped on every substantive change.**

Contents

1. Executive summary — what GRIT is
2. How LLM inference works (the mental model)
3. What an agent is, and why it stresses serving differently
4. The ecosystem, the seam, and the product/PM lens
5. The literature, annotated (foundations → frontier)
6. The tools + the open-source landscape (repo crawl)
7. The GRIT project — problem, metric, pilot, threats, status
8. For the NVIDIA inference-PM role
9. Open questions, the benchmark↔production gap, who to talk to
10. Next actions · reading queue · unverified citations

1. Executive summary — what GRIT is

GRIT (GPU Reuse under Interleaved Traffic) is a **measurement-and-characterization** study — not a new system, scheduler, or kernel. It asks a single question: *when chat and agents share one serving engine, how much of an agent's cache reuse actually survives, and what does the lost reuse cost per task the agent completes?*

An **agent** is a stateful loop (model → tool → model → ...) whose context *tends* to grow (absent compaction) and which re-enters the model many times against a mostly-shared prefix. Prefix caching *should* make each re-entry nearly free — **if the cached prefix survives to the next step**. Whether it does, under realistic *mixed* traffic on a *bounded* cache, is contested in the 2026 literature, and that disagreement is the opening.

GRIT quantifies the **locality gap** (how far *realized* reuse falls below *available* reuse), isolates how much of the drop is caused by **multi-tenant interleaving** vs. the agent's own context churn, and prices it as the **"Cost of Grit"** — cost per *verified task*, on open infrastructure (vLLM/SGLang) with public benchmarks (SWE-bench Verified, τ^2 -bench). It **plans** two deliverables (it is pre-data): **C1**, the measurement (the locality-tax curve), and **C3**, a *planned* mixed, cost-labeled, OpenTelemetry-format serving trace + harness. **Discipline: measurement, not mechanism** — the mechanism space (eviction/TTL/sharing/scheduling) is crowded and well-funded; defensibility is the *bundle* (open infra + mixed traffic + interleaving isolated + cost tied to verified work + the released trace), anchored by a defensible causal primitive (§7).

Status: pre-data. The harness is a scaffold; a set of design issues gate GPU spend (§7). The headline metric was sharpened in June 2026 from a vague “share of the gap” into a same-stream counterfactual after an adversarial review — see §7.

2. How LLM inference works (the mental model)

You cannot reason about agent serving without the inference roofline. Read this section once; everything later hangs off it.

Two regimes: prefill vs. decode

Generating from a transformer has two phases. **Prefill** reads the prompt and is **compute-bound** (lots of matrix math, high arithmetic intensity). **Decode** generates output one token at a time and is **memory-bandwidth-bound** (each step mostly reloads weights + the KV cache from GPU memory). The single most important consequence: in decode, wall-clock is dominated by reloading the weights (and KV) from HBM, not by the per-token math — so once you have paid that fixed memory cost, adding more *sequences* to the batch (or verifying several speculative tokens at once) is nearly free per token. That is why batching and speculative decoding help, and why **the memory wall**, not FLOPs, sets decode wall-clock. (Attention work does still grow with sequence length — the “nearly free” is about amortizing the weight read, not zero cost.)

The KV cache — the resource this whole project is about

Attention lets each token attend to all previous tokens; to avoid recomputing that history every step, the model stores the per-token **key/value** tensors — the **KV cache**. It is large, it lives in scarce GPU memory (HBM), and it is the central managed resource of a serving engine. Everything in the literature is either (a) moving work across the compute/bandwidth line or (b) managing the KV cache — its **size** (MQA/GQA/MLA, quantization), **reuse** (prefix/radix/prompt caching), **eviction** (LRU, H2O, TTL), or **placement** (offload, disaggregation).

Prefix caching — the lever that makes multi-turn cheap

Prefix caching reuses the KV of a shared prefix (system prompt, tool definitions, conversation/agent history) across requests or turns, so the shared part is not re-prefilled. **RadixAttention** (SGLang) generalizes this with a radix tree that shares prefixes *by content* across all in-flight requests. This is exactly the lever an agent loop depends on — *unless the prefix is evicted before the next step*. GRIT measures how often that “unless” bites.

Batching, goodput, and disaggregation

- **Continuous (iteration-level) batching** (Orca) interleaves requests at the token step, not the request boundary — the basis of modern throughput.
- **Goodput** (not raw throughput) is the SLO-meeting metric: useful work per second that actually meets latency targets. GRIT’s cost-per-verified-task is the *economic dual* of goodput (useful work per *resource*, not per second).
- **Prefill/decode disaggregation** (DistServe, Splitwise, Mooncake) splits the two phases onto different GPUs because colocating compute-bound prefill with bandwidth-bound decode causes mutual

slowdown. Agent loops re-trigger prefill constantly, so this tension is central to the workload.

The five things to be able to explain

(1) the roofline / arithmetic intensity; (2) the memory wall in decode; (3) goodput vs throughput; (4) prefill/decode interference; (5) “everything is the KV cache” — size, reuse, eviction, precision, placement. If you can place a technique on the roofline and say which KV property it touches, you understand it.

Key serving terms you'll also meet (not all expanded here — ask for any): **TTFT** (time-to-first-token), **ITL / TPOT** (inter-token latency / time-per-output-token), **E2E latency**; **queueing / admission** (a request waits for a batch slot + KV budget); **parallelism** — tensor / pipeline / expert / sequence (how one model is split across GPUs); **MoE** (sparse expert routing) and **MLA** (DeepSeek's latent-KV compression); **context compaction** (summarize/truncate to fit); and **cache-key correctness** (a prefix hit must be the *same* tokens, tokenizer, and sampling params, or it is unsafe).

3. What an agent is, and why it stresses serving differently

A **chat request** is typically one prompt → one short stream (it *can* be multi-turn with persistent prefix reuse, but with a shorter, less monotonically-growing prefix). An **agent** is a stateful loop — the model proposes an action, a **tool** runs (code execution, search, a file edit), the result is appended, and the model is re-entered — for tens to ~200 steps. Four properties *tend* to make this a different serving workload:

- **Monotonically growing context.** Each step appends; the prefix the model re-reads gets longer and longer.
- **Heavy re-entry against a shared prefix.** The same long history is re-read every step — enormous *potential* reuse, which is exactly what caching should capture.
- **Tool-call gaps.** Between model turns the agent waits on a tool. The GPU may idle (single-tenant) or serve others (multi-tenant), and the agent's KV sits idle and evictable.
- **Long-lived KV state.** The cache must survive across tool gaps and across other tenants' traffic to pay off.

*These are tendencies, not laws. Real agents also **summarize, truncate, edit, branch, and spawn sub-agents** — each of which resets or forks the prefix and breaks the "monotonic growth" picture — and chat can be multi-turn. GRIT measures the regime where the tendencies hold (long-horizon, growing-prefix loops); compaction/branching are V2 axes.*

Agent patterns (the loops themselves)

ReAct — Yao et al., ICLR 2023 · arXiv 2210.03629 ✓

Interleaves *reasoning* traces with *acting* (tool calls). The default agent loop and GRIT's pilot methodology.

Reflexion / Toolformer / ReWOO (Plan-and-Execute) / Tree of Thoughts — arXiv 2303.11366 / 2302.04761 / 2305.18323 / 2305.10601 ✓

Reflexion adds self-critique memory; Toolformer learns when to call tools; ReWOO decouples planning from execution (a different KV-reuse profile); ToT searches over reasoning branches (fan-out). These define the space of loops a serving study must keep in mind even if the pilot fixes one (ReAct).

Model Context Protocol (MCP) — Anthropic, open standard ✓ (context)

The emerging tool-interface standard — the “pauses” in the loop are MCP/tool calls.

Why this matters to GRIT: the high re-entry + growing prefix is the source of *available* reuse; the tool gaps + long-lived KV are where eviction under shared load destroys *realized* reuse. The gap between the two is the whole study.

4. The ecosystem, the seam, and the product/PM lens

The interactive version of this map is <docs/ecosystem-map.html> (a systems-paper “Figure 1”). In words, the stack runs left-to-right:

- **Clients & agents** — coding agents (Cursor, Claude Code, Codex, OpenHands), web/computer-use agents, and **chat/RAG traffic** (the co-tenant in mixed serving).
- **Orchestration** — LangGraph, CrewAI, the OpenAI Agents SDK drive the loop. Crucially they are **workflow-agnostic to the engine**.
- **Open serving engines** — vLLM, SGLang — plus **KV infra** (Mooncake, LMCache, llm-d) and a crowded field of **agent-aware mechanisms**.
- **Vendor / hardware** — NVIDIA Dynamo (KV-aware router + agent hints), AMD/TPU.
- **Measurement** — Cost-of-Pass (the anchor), AA-AgentPerf, InferenceX, and the task benchmarks.

The seam (the *hypothesized* root cause)

By default the orchestrator and the engine are decoupled: the engine sees a **flat stream of independent requests** and is blind to the agent loop, the coming tool pause, and cross-request prefix sharing (Cortex’s word: “**workflow-agnostic**”). Under mixed traffic on a bounded cache, that blindness is where an agent’s realized reuse falls below its available reuse. The 2026 frontier is a race to break the seam, split into two families:

- **Declared** (the agent *tells* the engine its structure): Dynamo `nvext.agent_hints`, KVFlow (Agent Step Graph), Helium (query plan), HexAGenT (online-revealed DAG), Cortex (call graph + SLO slack), Autellix/ThunderAgent (workflows-as-programs), Sutradhara (explicit co-design).
- **Inferred** (the engine *guesses*, no signal): Continuum/CacheTTL (a TTL heuristic), GoodServe (predicts output length + GPU state at the router).

GRIT’s wedge: nobody has *measured*, on open infra under realistic mixed traffic, how much reuse is lost when this awareness is absent vs. present — the “**value of awareness**.” **Severity caveat:** RadixAttention already shares prefixes by content and good routing recovers a lot — so the seam bites hardest under **bounded cache + interleaving + long horizon**, and may be minor when well-provisioned. (Also: workflow-blindness is *one* candidate mechanism — ordinary capacity pressure, routing, block-size, and scheduling can lose reuse without it; H1 must isolate it.) If “just add cache” recovers most reuse, the measured value of awareness is small — itself a publishable result.

Personas & their pains (who acts on the number)

Persona	Pain	What GRIT gives them
Platform / serving engineer	can't explain why agents cost ~5x chat under shared load	the locality-tax attribution
Agent product builder	\$/token looks fine but \$/completed-task explodes at long horizons	the Cost-of-Grit curve
ML-systems researcher	no public mixed, cost-labeled agent trace; can't compare engines fairly	the released trace (C3)
Eng leadership / FinOps	unpredictable bills; no cost-per-successful-task meter	a budgetable meter
Inference-vendor PM (NVIDIA)	no open, neutral measurement of where agent cost actually goes — hard to prioritize what to build	the open meter as a north-star signal (→ §8)

User journeys (the pain in motion)

1. **The surprise bill** (agent startup): usage scales, the token bill 4xs, per-token price is unchanged — but **cost per completed task exploded at long horizons** and nothing shows why. → needs the cost-of-grit meter.
2. **The platform team's blind spot** (infra eng): they co-locate chat + agents to save GPUs; agent latency/cost spikes; they suspect eviction, but the engine exposes only an **aggregate** hit rate, not “did chat evict *this agent's* cache.” → needs per-tenant attribution + the locality-gap metric.
3. **The apples-to-oranges comparison** (researcher): wants vLLM vs. SGLang for agents, but there is no public mixed trace and no standard cost-per-task metric, so every paper measures differently. → needs the open trace + the metric.

Pain → opportunity (the product / business view)

Pain (market)	Opportunity	Who captures it	GRIT?
Cost grows super-linearly with horizon, hidden by \$/token	the meter : cost-per-verified-task (the locality-tax curve)	researchers + FinOps + vendors	✅ C1 (paper)
Cache reuse collapses under shared / bounded cache	quantify the realized-vs-available gap; the value of awareness	serving teams	✅ C1 / H1
No per-tenant cost / eviction visibility	instrumentation + attribution methodology	platform teams, vendor (Dynamo)	partial (paper measures; product builds)
Orchestrator↔engine seam (declared vs inferred)	a hint contract / standard	vendors (Dynamo), llm-d / Gateway	⚠ pre-empted (Dynamo); measure its value
No public agentic + cost-labeled trace	release the trace	the commons	✅ C3 (artifact)
Can't compare engines/configs fairly	reproducible open-infra benchmark + metric	the commons	✅ bundled with C1/C3

Where GRIT plays vs. the product side: the **open measurement** (the meter + the trace, rows marked ) is the *paper*. The **productized** versions — building the meter into a serving stack or a FinOps SaaS, shipping the hint contract — are the **product / internship** side (the “C5” twin), out of the paper’s scope but the bridge to the NVIDIA PM role (§8). Full companion: `docs/ecosystem-and-product-map.md` (relational graph + mindmap + personas/journeys) and the interactive `docs/ecosystem-map.html` (Figure 1).

5. The literature, annotated

Every entry below is  in the ledger. Paragraph summaries for the load-bearing papers; grouped one-liners for the rest. **Relevance-to-GRIT** is highlighted where it matters.

5a. Foundations — the inference canon

Attention Is All You Need — Vaswani et al., NeurIPS 2017 · arXiv 1706.03762 

The transformer. The KV cache exists because of attention; the locality gap is a statement about how well that cache is managed.

Efficiently Scaling Transformer Inference — Pope et al., MLSys 2023 · arXiv 2211.05102 

The clearest analytical account of the inference roofline (compute- vs bandwidth-bound), MFU, and why attention variants scale. Read first to build intuition.

Orca; PagedAttention / vLLM; Sarathi-Serve — OSDI 2022 / SOSP 2023 (2309.06180) / OSDI 2024 (2403.02310) 

The serving-systems baseline: continuous iteration-level batching (Orca); OS-style paging of the KV cache so it isn’t contiguous (vLLM — our open baseline); chunked prefill that interleaves long prompts with ongoing decode (Sarathi-Serve, also a confound for latency).

SGLang / RadixAttention — Zheng et al., NeurIPS 2024 · arXiv 2312.07104 

Radix-tree prefix sharing across requests + a compressed-FSM for structured output. **The most relevant cache story to GRIT** — content-based sharing is the best *content-based* reuse baseline the study measures against (the true ceiling is the exact-prefix *resident-oracle* replay, §7 — RadixAttention is a bounded implementation, not the ceiling). Note: SGLang’s tree eviction differs from vLLM’s paged LRU, so the two engines are **not** directly poolable (a stratification variable, not a shared axis).

Disaggregation: DistServe / Splitwise / Mooncake — 2401.09670 / 2311.18677 / 2407.00079 

Split prefill and decode across GPUs/machines for goodput; Mooncake is KVCache-centric (the stack behind Kimi) and the basis of a public agent trace (see 5e).

KV size, reuse, eviction, precision — MQA 1911.02150 · GQA 2305.13245 · Prompt Cache 2311.04934 · StreamingLLM 2309.17453 · H2O 2306.14048 · KIVI 2402.02750 · FlashAttention 2205.14135 (+2/3) · FlashInfer 2501.01005 

Shrink the cache (MQA/GQA one/few KV heads; KIVI 2-bit quantization); reuse it (Prompt Cache modular reuse); evict it (StreamingLLM attention sinks + window; H2O heavy-hitter eviction — the academic ancestors of agent-aware eviction); compute attention IO-efficiently (FlashAttention; FlashInfer kernels).

Parallelism, quantization, speculative decoding — Megatron 1909.08053 · GPipe 1811.06965 · ZeRO

1910.02054 · Ring 2310.01889 · LLM.int8 2208.07339 · GPTQ 2210.17323 · SmoothQuant 2211.10438 · AWQ 2306.00978 · SpecDecode 2211.17192 · Medusa 2401.10774 · EAGLE 2401.15077 ✓

The composable axes (data × tensor × pipeline × sequence/expert), weight/activation quantization (GRIT holds the model + quantization fixed), and draft-and-verify decoding. Cite for concepts; not the focus.

5b. Agentic workload characterization (the closest prior art)

Agentic AI Workload Characteristics — Yuan, Nayak, Kundu, Talati (UIUC / Gimlet / Intel) · arXiv 2605.26297 ✓

Traces ReAct agents across five benchmarks; finds agentic serving is **decode-dominated with very high KV reuse**, plus a temporal **read/explore** → **execute/write** tool-use structure. **The “high available reuse” pole of GRIT’s contradiction** (caching should win big).

Sutradhara — Biswas et al. (Microsoft Research / M365) · arXiv 2601.12967 ✓

Synthetic requests at production scale (not real customer traffic — important). Finds tool calls are 30–85% of time-to-first-token and **KV hit rates collapse despite reuse**; proposes orchestrator-engine co-design. **The “realized collapse” pole** — but it attributes collapse to intra-request churn + eviction, **not** specifically to multi-tenancy, which is precisely the opening: nobody has cleanly isolated the multi-tenant-interleaving contribution on open infra.

KVCache-in-the-Wild — SJTU/Alibaba, USENIX ATC’25 · arXiv 2506.02634 ✓ (closest pre-empt)

Production-trace characterization of KV reuse including an **ideal-hit-ratio-vs-cache-size** curve (≈ realized-vs-available) on real mixed traffic. **Pre-empts the characterization half** — but on a *closed* stack, no agent tool-gap isolation, no cost-per-task. GRIT’s wedge = open infra + agent + cost-per-verified-task.

ServeGen; CPU-Centric Agentic AI; Agent Memory; Compiling Agentic Workflows — 2505.09999 (NSDI’26 claimed) · 2511.00739 · 2606.06448 · 2605.22502 ✓

ServeGen: real prod characterization (billions of req) + a generator, but *non-agentic* data. CPU-Centric: host-CPU tool processing can dominate latency (so “tool gap” ≠ “GPU idle”). Agent Memory (Omri/Tambe, Stanford): first systems characterization of agent *memory* — a close characterization competitor on a different axis (memory state vs KV-locality/cost). Compiling-into-weights: near-frontier quality at ~1/100 cost — the boundary that scopes GRIT to non-compilable workloads.

5c. Cache / scheduling mechanisms for agents (CROWDED — mechanism paper is dead)

Continuum/CacheTTL (2511.02230, KV TTL across tool gaps) · KVFlow (2507.07400, workflow-aware eviction via an Agent Step Graph) · KVCOMM (2510.12872, training-free cross-agent KV reuse) · Helium (2603.16104, workflows as query plans) · Autellix (2502.13965, agents-as-programs engine) · ThunderAgent (2602.13692, program-aware scheduler, 1.5–3.6× vLLM) · SAGA (2605.00528, workflow-atomic scheduling on vLLM; predicts cross-tool-call reuse to within 1.31× of Bélády) · HexAGenT (2605.16637) · Cortex (2510.14126) · SparseX (2606.01751) · TokenDance (2604.03143) · ScaleSim (2601.21473) · Aragog (2511.20975) · AWO (2601.22037) · CONCUR (2601.22705) · plus a 2026 wave: TokenCake (2510.18586), RelayCaching (2603.13289), PrefillShare (2602.12029), PPD (2603.13358), CacheFlow (2604.25080), Halo (2509.02121), PBKV (2605.06472), AgentServeSim (2606.09613), Agent.xpu (2506.24045), WRP (2603.21354). All ✓.

Why GRIT is not a mechanism paper: every one of these *builds* the fix. GRIT instead **measures the value** of awareness on open infra — and uses these as the “present awareness” comparison points. **Adjacent (not agent-specific):** FlashMemory-DeepSeek-V4 / Lookahead Sparse Attention (2606.09079, Tencent) — a learned Neural Memory Indexer predicts query-critical KV chunks (~13.5% kept); *intra-request* long-context sparsification, DeepSeek-V4-coupled, preliminary — prior art for the phase-structure hypothesis (H3), does not pre-empt C1.

5d. Measurement & cost (the angle adjacent to GRIT)

Cost-of-Pass — Erol, El, Suzgun, Yüksekönül, Zou (Stanford) · arXiv 2504.13359 · OpenReview vC9S20zsgN (ICLR’26 submission; acceptance unconfirmed) ✓

Formalizes **cost-of-pass** = the expected monetary cost of generating a *correct* solution, built on Farrell’s productive-efficiency theory; defines a frontier cost-of-pass across models. **The academic anchor for GRIT’s denominator.** GRIT moves it from the API black box into open-infra internals and ties its cost term to the locality gap — that move, not the metric name, is the novelty. Cite it; do not imply the metric is new.

GoodServe — Du et al. (SJTU/CUHK-SZ) · arXiv 2605.16867 ✓

Agentic **goodput** = E2E-SLO completions/s over heterogeneous GPUs. Router-layer: **predicts output length** (a MoE of MLPs, with task type as a *feature*) + GPU serving state, then routes “predict-and-rectify” (no hints). No cost-to-success, no released trace. **The throughput-side dual of GRIT’s cost-per-verified-task** — they optimize, GRIT measures + attributes to cache locality.

Don’t Break the Cache; More with Less; EET; Efficient Agents; HAL; SWE-EVO; TokenPowerBench

— 2601.06007 · 2510.16786 · 2601.05777 · 2508.02694 · 2510.11977 · 2512.18470 · 2512.03024 ✓

Don’t Break the Cache: quantifies prompt caching for agents at the *provider-API black box* (the trap GRIT avoids by measuring open-infra internals). More with Less: cost-per-patch (~\$5.85 avg; 41–58 median turns) on the capability side. EET: experience-driven early stopping (–32% cost, ≤0.2% resolution loss — “when to stop gritting”). Efficient Agents: efficiency–effectiveness tradeoff. HAL: accuracy–cost Pareto. SWE-EVO: multi-PR tasks + a partial-progress “Fix Rate.” TokenPowerBench: J/token energy (the energy numerator). Latency-Reliability-Cost (2605.23929), Efficiency Frontier (2605.23071), Price of Progress (2511.23455), Observation-Masking (2508.21433, a cost confound), SWE-Pruner (2601.16746) round out the set.

5e. Benchmarks & public traces

Task benchmarks: SWE-bench Verified (MIT; the de-facto coding-agent set), SWE-bench Pro (2509.16941, contamination-resistant), SWE-bench-Live, SWE-EVO (2512.18470), AppWorld (2407.18901), τ^2 -bench (Sierra; tool-agent-user), GAIA, BFCL. **GRIT’s pilot runs τ^2 + SWE-bench Verified.**

Public serving traces (the C3 landscape):

Trace	real?	agentic?	cost-labeled?	role for GRIT
BurstGPT (2401.17644 ✓)	yes (Azure-OpenAI, 213	no (chat)	no	the realistic chat co-tenant

Trace	real?	agentic?	cost-labeled?	role for GRIT
	days)			
Azure LLM Inference Traces	yes	partial (coding)	no	co-tenant + arrival dynamics
vLLM × Mooncake corpus	yes	yes (agent-only)	no	replay agent cache behavior
GAIATrace (2606.01725 ✓)	paper yes; release pending	yes (GAIA)	no	token-level trace + Vidur-Agent sim — open-sourced “upon publication”, not yet available

The empty cell remains: none is *mixed chat+agent + cost-labeled + open-infra + OTel*. So C3’s niche holds — but GRIT should **ingest** BurstGPT/Azure (chat) + vLLM×Mooncake/GAIATrace (agent) rather than synthesize, which raises realism and reframes C3 as “the missing combination.” ⚠️ **Locate the primary URL for the vLLM×Mooncake figures before citing them.**

5f. Industry / vendor (motivation only — never cited as research evidence)

NVIDIA Dynamo — KV-aware router (global KV-block index + a cache-overlap-plus-load routing cost) + agent hints (under `nvext`, sibling fields `agent_hints {priority, osl, speculative_prefill}` and `cache_control {ephemeral, ttl}`); NVIDIA cites 85–97% **managed-provider Claude-Code** cache hits as motivation (*not* a Dynamo self-hosted benchmark, and not infinite-cache “available”); no multi-tenant interleaving analysis, no released traces. **AA-AgentPerf** (Artificial Analysis) — agents-per-megawatt under SLO tiers on real coding trajectories; test set *closed*; cost-per-success is future work. **InferenceX / InferenceMAX** (SemiAnalysis) — open \$/token / throughput, not agentic. **NemoClaw / OpenShell** — autonomous-agent blueprints + sandboxed runtime.

Net for GRIT: C4 (a hint spec) is **pre-empted** by Dynamo; re-scope to measuring the value of existing hints. C1 (cost-per-verified-task tied to cache) and C3 (a mixed, cost-labeled, open trace) stay distinct and are *strengthened* by the fact that the vendor numbers are realized-only and the test sets/traces are closed.

5g. Newest entrants (June-2026 sweep)

GORGO (2602.11688 ✓) — cross-region load balancing that maximizes KV-cache reuse *while* minimizing network latency (the GORG**O**-*proxy* is ~2.5× faster median TTFT vs. prior routers — not an unconditional result); the **placement** axis (where reused KV lives across regions). **Pie** (SOSP’25, ACM DL 10.1145/3731569.3764814 ✓) — a programmable serving system exposing *inferlets* with explicit KV allocation/reuse + agentic-workflow evaluation; it directly attacks the orchestrator↔engine boundary, so treat it as **close architectural prior art** (compare GRIT’s intervention + evaluation against it), not merely a “signal.” **KVFlow** is a confirmed **NeurIPS 2025** poster. Adjacent (RAG & cross-DC frontier): ProphetKV (2602.02579 ✓, selective recompute for RAG), Prefill-as-a-Service (2604.15039 ✓, cross-datacenter KVCache). **Watch (web-found, not deep-read):** TimelyLLM (time-sensitive serving for physical-I/O agents); Graph-CoT multi-agent serving (2511.01633); Agent Memory Below the Prompt (2603.04428 — a *distinct* edge/Q4-KV paper, not the Stanford 2606.06448).

6. The tools you will actually use

- **vLLM** (`--enable-prefix-caching`) — the open baseline engine. Exposes `vllm:prefix_cache_queries` / `vllm:prefix_cache_hits` Prometheus counters and per-request `num_cached_tokens` — GRIT’s realized-reuse source.

- **SGLang** (RadixAttention) — the second engine; the best *content-based* reuse baseline (not the oracle ceiling).
- **KV infra:** LMCache (re-exposes cache metrics via the vLLM `/metrics` endpoint), Mooncake / llm-d (distributed KV pools/offload).
- **Observability:** Prometheus `/metrics`, vLLM `SchedulerStats + LLM.get_metrics()`; the OpenTelemetry gen-ai semantic-conventions effort (proposed `gen_ai.server.kv_cache.*` — not stable yet). **Caveat (see §7):** per-tenant *eviction attribution* is not exposed off-the-shelf and needs a validated engine patch.
- **Benchmark harnesses:** SWE-bench's `FAIL_TO_PASS` oracle; τ^2 -bench's success check; HAL for cost-aware scoring.
- **The GRIT harness:** `05-experiments/pilot/harness/trace_schema.py` (the measurement contract — runs/smoke-tested) and `run_pilot.py` (the orchestration skeleton with co-design seams). **Wrong-layer tools:** GPU profilers (Nsight Compute/Systems) and reuse predictors (Tencent FlashMemory) cannot source per-tenant eviction attribution — it is an engine block-manager fact (see `docs/observability-and-power.md`).

The open-source landscape — GitHub stars (crawled 2026-06-14)

A rough map of where the energy is. Agent *harnesses/frameworks* dwarf serving engines in stars (they are user-facing), but the serving/KV layer is where GRIT's measurement lives — the less-crowded side of the stack.

Repo	★ (k)	What it is
Significant-Gravitas/AutoGPT	185	the original autonomous-agent project
ggml-org/llama.cpp	117	CPU/edge LLM inference in C/C++
browser-use/browser-use	99	web / computer-use agent
vllm-project/vllm	83	open serving engine — GRIT baseline
OpenHands/OpenHands	77	coding-agent harness
FoundationAgents/MetaGPT	69	multi-agent framework
cline/cline	63	autonomous coding agent (IDE/CLI/SDK)
microsoft/autogen	59	agentic-AI programming framework
crewAIInc/crewAI	54	role-playing multi-agent orchestration
Aider-AI/aider	46	terminal AI pair-programming
langchain-ai/langgraph	35	agent orchestration graph
langfuse/langfuse	29	LLM/agent observability (OTel-native)
sgl-project/sglang	29	open serving engine — RadixAttention; GRIT
openai/openai-agents-python	27	OpenAI Agents SDK
SWE-agent/SWE-agent	20	issue-fixing agent (NeurIPS'24)
QwenLM/Qwen-Agent	17	Qwen agent framework (GRIT's model family)
NVIDIA/TensorRT-LLM	14	NVIDIA inference optimizer

Repo	★ (k)	What it is
huggingface/text-generation-inference	11	HF serving (TGI)
LMCache/LMCache	9.0	KV-cache layer (offload / reuse)
ai-dynamo/dynamo	7.3	NVIDIA datacenter inference framework
kvcache-ai/Mooncake	5.6	KVCache-centric serving (Kimi); the agent-trace source
SWE-bench/SWE-bench	5.2	the coding-agent benchmark (GRIT arm)
llm-d/llm-d	3.4	K8s-native distributed inference
sierra-research/tau-bench	1.3	tool-agent-user benchmark (τ^2 -bench family; GRIT arm)

7. The GRIT project

Claim-status convention (read §§7–8 with this)

[fact] = externally verified · **[plan]** = proposed design, not built · **[gate]** = unresolved blocker. **Everything about GRIT’s own results is [plan] or [gate] — the project is pre-data, with zero collected measurements.** The locality-tax counterfactual, the metric, the trace (C3), and the pilot are **[plan]**; the blockers below are **[gate]**. Only external prior work and vendor/product facts are **[fact]**.

The problem, precisely

Two poles that don’t meet: high *available* reuse (2605.26297) vs. *realized* collapse (Sutradhara, synthetic, intra-request) — because **no one has isolated the multi-tenant driver on open infrastructure**. Under realistic *mixed* chat×agent traffic on a *bounded* open-infra cache, GRIT asks: (1) how far realized falls below available (the **locality gap**); (2) how much of the drop is **interleaving** vs. intra-agent churn; (3) what it costs **per verified task** (the Cost of Grit). Root cause: the workflow-agnostic **seam** (§4).

Contributions

	Contribution	Status
C1 (lead)	the realized-vs-available cache-locality gap quantified on open infra, priced as the “Cost of Grit” (cost per verified task)	pilot pending
C3 (artifact)	a <i>planned</i> mixed chat×agent, cost-labeled, OpenTelemetry-format serving trace + harness (not yet collected; schema/licensing/redistribution gates open)	[plan] — bundled with C1

C2 (cache-aware context-edit policy), C4 (hint interface — pre-empted by Dynamo, re-scoped to value-of-hints), C5 (the product/PM metering twin — out of scope for the paper), C6 (harness-conditional benchmarking) are secondary/parked. Killed ideas live in 04-ideas/graveyard.md (a generic mechanism paper; naive “agents ≠ chat” characterization; “first public agent trace” as a headline).

The metric — sharpened after adversarial review (the make-or-break)

(Proposed headline, **pending co-author ratification** — STATUS.md, the pilot README, matrix.yaml, and the harness still use the per-iteration diagnostic form; this is the redesign, not yet adopted repo-wide.) The

denominator is **cost per verified task** (binary success: SWE-bench `FAIL_T0_PASS` / τ^2 -success) over a **frozen task population** with a pre-registered retry policy, counting the cost of **failed attempts** (that is where grit's cost lives). Numerator: GPU-seconds (primary — *price-neutral*, but NOT hardware-neutral: a result is only comparable with accelerator, precision, kernels, and engine version pinned), plus \$ and — where power telemetry is validated — Joules. Always a **curve** over horizon $\times$ cache policy $\times$ tenancy, never a scalar. Partial credit (Fix Rate) is a *separate supplementary* view, not the headline denominator; do not pre-select “solvable” tasks.

The locality tax must be a same-stream counterfactual (not a “share of the gap”)

A June-2026 adversarial review correctly objected that “the fraction of cost attributable to eviction” is **not identified** — the \$/token-vs-\$/task gap absorbs context growth, iteration count, failures, tool cost, queueing, and model behavior, so a cache-hit-rate delta cannot, alone, claim a share of GPU-seconds. **The fix (the real C1 primitive)**: record the agent trajectory *once*, freeze the exact token/block stream, and replay it twice under the engine schedule — (a) the **bounded** cache that shipped, (b) a counterfactual where every reuse-eligible prefix block is **resident** (oracle cache) — holding model, decode, ordering, and offered load fixed. **locality tax** $\equiv \Delta$ GPU-seconds (bounded – resident) per verified task. If that counterfactual is not realizable on stock engines, it **gates C1**. Two scope-matched estimands (lineage-only; global-stream) are reported separately, because engine-realized hits can include other tenants' blocks (so “realized $\leq$ available” only holds on a matched scope).

Hypotheses (each falsifiable, with a kill criterion)

- **H1** — the locality gap is real on open infra, and multi-tenant **interleaving** is a distinct driver (separable from intra-agent churn). *Clean test*: same request multiset/arrivals/budget/load; vary only ordering and unaware-vs-(oracle/hinted) retention — not isolated-vs-mixed, which changes composition.
- **H2** — the Cost of Grit grows **super-linearly** with horizon and is cache-policy-sensitive. *Kill*: linear and policy-insensitive $\rightarrow$ no tax story.
- **H3** — tool-gap timing has exploitable **phase structure** static eviction gets wrong (shown in simulation, not built). *Kill*: tool-gap timing is effectively random.
- **H4** — a small public **trace + harness reproduces H1–H2** (artifact-as-contribution).

The pilot (first, cheap, kill-fast)

ReAct $\times$ $\{\tau^2\text{-bench, SWE-bench Verified}\} \times \{\text{isolated, mixed}\} \times \{\text{LRU, retain-during-tool}\} \times 50 \times 3 \approx 1,200$ trajectories, $\sim 1\text{--}2$ weeks, $\sim \$500\text{--}600$ of rented H100. τ^2 is the cheap tool-gap probe; SWE-bench Verified is the long-horizon arm where the tax should be visible (a τ^2 -only pilot risks a false-negative kill of H1). Model fixed at Qwen2.5-Coder-32B (the model-vs-system control). **Caveat**: this is the *current scaffold* matrix; per the H1 “clean test” above and pilot issues #20–22, isolated-vs-mixed changes composition (not a clean interleaving contrast) and `retain-during-tool` is not a stock knob (building it = building a Continuum/Dynamo-style mechanism) — the identifying design is the **same-stream counterfactual** with stock or an explicitly-named existing retention, and the matrix is being revised toward it.

The two BLOCKERS and how they get solved (docs/observability-and-power.md)

(1) Can we see the cache per request? The realized numerator is gettable today (per-request `num_cached_tokens` ; prefix-hit counters). The genuinely missing piece is **per-tenant eviction attribution** (“did chat evict *this* agent’s blocks?”). vLLM’s eviction telemetry carries lifetime/idle/reuse-gap — **not** block-id/owner/tenant — so attribution needs a **validated, version-pinned block-manager patch** (not “a few lines”); a footprint-matched proxy is a weaker, suggestive fallback. This **gates C1** and is the first thing the spike checks. **(2) Is the denominator big enough?** A bare ReAct loop may resolve only single-digit % of SWE-bench Verified → too few verified tasks per cell. Use a stronger scaffold / solvable subset / τ^2 as primary; pre-register a minimum-successes-per-cell floor.

Replay ≠ instrumentation. Recovering *available* reuse offline is gated by what the harness *logs* (the recorded stream), not by engine internals — the easier half. **Statistical soundness is NOT yet established:** there is no variance estimate, MDE, or power result; the 1,200-trajectory matrix is *not* “power-sized” — run a micro-pilot, estimate task- and run-level variance, then pre-register a cluster-aware power calc (bootstrap over *tasks*, ~50 effective, not trajectories).

Threats to validity (the methods-section commitments)

Model-vs-system boundary (hold the model fixed; the #1 threat); prove interleaving, don’t assume it; statistical rigor + pre-registered effect size; benchmark licensing/currency + version pinning; artifact-evaluation hygiene; trace-release PII/legality. **Plus the confounds added after review:** saturation/provisioning regime; chat-tenant SLO harm; synthetic-mixture validity (drive the chat co-tenant from a real trace); instrumentation/engine-patch perturbation; quantization/block-size/routing/placement; timeout censoring; scaffold selection; multiple comparisons; benchmark-oracle error. Full list: docs/threats-to-validity.md.

Honest status

Pre-data; the metric/trace contract is under active redesign and the design issues gate GPU spend. An external adversarial (Codex) review in June 2026 returned a provisional *reject* with one decisive risk: **no defensible counterfactual connecting lost reuse to cost-per-task**. That objection is now made *operationally testable* (not yet answered) by the same-stream oracle-vs-bounded definition above; **C1 remains unestablished until that resident-cache replay is built and validated**. The window is closing (the field is converging on this seam); defensibility is the bundle + open reproducibility + the counterfactual primitive, not any single number.

8. For the NVIDIA inference-PM role

This section reframes everything above for the **product** seat on an inference team. The technical fluency (§2–§6) is table stakes; this is the layer on top — what the product owns, who the competition is, and how a measurement like GRIT informs the roadmap. (GRIT is the *open* measurement paper; the productized version is the work side — the knowledge transfers directly, the artifacts stay separate.)

NVIDIA Dynamo in depth (the product you will reason about)

Dynamo is NVIDIA’s datacenter-scale, disaggregated inference-serving framework. The agent-relevant pieces:

- **KV-aware router [fact]** — a global KV-block index + a *cache-overlap-plus-load* routing cost (route to the replica holding the most of the prefix, balanced against load — not pure max-overlap). NVIDIA cites 85–97% **managed-provider Claude-Code** cache hits as motivation — *not* a Dynamo self-hosted benchmark, and not infinite-cache “available.”
- **Agent hints [fact, shipping]** — under `nnext`, the *sibling* fields `agent_hints` {priority, osl (output-seq-length), speculative_prefill} and `cache_control` {ephemeral, ttl}: the **declared** side of the seam (this is what pre-empts a “propose a hint spec” paper).
- **Disaggregated prefill/decode [fact]** + a distributed KV block-manager / transfer layer (conceptually adjacent to Mooncake / LMCache — confirm Dynamo’s exact components before quoting).
- **Roadmap [gate — UNVERIFIED]:** an agentic trace-replay / trajectory-interchange capability has been mentioned, but I could **not** confirm it in the Dynamo repo or roadmap — treat as a rumor to verify, not a fact. *If real*, NVIDIA would be productizing the trace/replay tooling GRIT builds (both validation and competition for C3).

What an inference-team PM actually owns

- **Prioritization under the seam:** which agentic pains (the persona table, §4) are biggest *and* most monetizable — and whether to invest in the **declared** (hints) or **inferred** (engine-side) contract.
- **Metering / observability as a product** (the “C5 twin”): per-tenant cost + locality attribution is a feature platform teams *plausibly* want — **validate via customer discovery** (the personas here are reasoned, not interviewed); GRIT would supply the methodology and, *once it has data*, the dollar value.
- **Positioning vs. benchmarks:** AA-AgentPerf (agents/MW) and InferenceX (\$/token) set the narrative; cost-per-*verified-task* is the missing, more customer-aligned frame.
- **Open-source posture:** vLLM / SGLang / LMCache / llm-d are both the substrate and the competition; the moat is KV-aware routing + hints + trace/replay + hardware. (Star table, §6, shows where community energy sits.)

How GRIT’s findings feed product decisions

- The **value-of-awareness** number = an *estimate (once measured)* of the \$ a customer could save from hints/routing → the ROI case for the declared contract.
- The **locality-tax curve** = a candidate customer-facing story for why agents can cost several× chat under shared load (*pending data*; eviction is only one component of total agent-vs-chat cost).
- The **per-tenant attribution** method = a candidate Dynamo observability feature.
- The **regime caveat** (the seam is minor when well-provisioned) tells you *which* customers the feature actually helps — bounded-cache, multi-tenant, long-horizon shops.

PM / ops tools to know (beyond the OSS engines): NVIDIA Triton Inference Server, NIM (packaged inference microservices), TensorRT-LLM (the optimizer beneath Dynamo), NIXL / KVBM (KV transfer + block management), GenAI-Perf (benchmarking), and the Kubernetes Inference Gateway (llm-d / Gateway-API routing). Plus the ops surface a PM owns: autoscaling / Planner, SLO & TCO, multi-tenancy isolation, fault tolerance, and deployment-operability. (Confirm exact product names/scopes against current NVIDIA docs before external use.)

Questions a PM on this team should be able to answer cold

Explain prefill vs. decode and the memory wall (§2). Why does an agent stress the KV cache differently than chat (§3)? What is the orchestrator↔engine seam, and what does Dynamo ship to close it (§4, §8)? Goodput vs. cost-per-task — why does \$/token mislead for agents (§7)? Who are the customers and what is each one's top pain (§4)? Where is the open-source competition and what is the moat (§6)? What would you measure to justify a routing-or-hints investment (§7, §8)?

9. Open questions, the benchmark↔production gap, who to talk to

Harness/developer needs & TTFT. The locality gap is a time-to-first-token story: a surviving prefix-cache hit skips prefill (low TTFT); an evicted prefix recomputes (high TTFT + cost). GRIT should report TTFT + tail-TTFT as first-class outputs, and promote “value of awareness” (C4 re-scope) to a measured axis. Deployment friction proper (cold start, autoscaling) is out of scope (the C5 product twin).

Benchmark↔production gap. Benchmarks are curated single tasks with synthetic arrivals; production is concurrent, bursty, multi-tenant. C3 answers this, but GRIT inherits the agent-distribution gap by running on benchmarks. **Fix:** drive the chat co-tenant from a *real* trace (BurstGPT/Azure) and name the residual gap; optionally replay a real agent corpus for the cache half.

Who to talk to (to understand agents in the wild). Inside NVIDIA: the Dynamo PM/eng + customer teams (real agent serving workloads). Serving vendors: Together, Fireworks, Baseten, Anyscale, Modal. The harnesses themselves: Cursor, Cognition/Devin, All Hands/OpenHands, Replit, Sourcegraph/Amp, Windsurf. Orchestration: LangChain/LangGraph, CrewAI. Observability (aggregate in-the-wild telemetry): LangSmith, Braintrust, Arize, Helicone. Trace authors: Moonshot (Mooncake), the BurstGPT/Azure + KVCache-in-the-Wild authors. Full writeup: [docs/discovery-and-gaps.md](#).

10. Next actions · reading queue · unverified citations

Immediate: (1) the co-design sync — settle the design issues (lead with the same-stream counterfactual, the trace fields, the cost-survivorship rule); confirm measurement is the lead; agree ownership + author order in writing. (2) Run the instrumentation + floor-rate **spike** before the full cell (it answers the circular “settle-before-spend” issues). (3) Then run the 1,200-trajectory pilot against the kill criteria. (4) Ingest BurstGPT/Azure as the real chat co-tenant. (5) Locate the vLLM×Mooncake primary URL.

Reading queue (priority): Cost-of-Pass (2504.13359) for the metric write-up; KVCache-in-the-Wild (2506.02634) + SAGA (2605.00528) as the closest pre-empts; Continuum/CacheTTL (2511.02230) + KVFlow (2507.07400) for the inferred/declared mechanisms; GoodServe (2605.16867) for the goodput dual; Agent Memory (2606.06448) + GAIATrace (2606.01725) for adjacent characterization/traces.

Still unverified — do NOT cite until checked: SideQuest (2602.22603); AutoLab (and its “+0.43” harness-ablation figure — do not quote); Inside the Scaffold; Self-Harness; Imcache-agent-trace; Agentix@NSDI.

Confirmed real since earlier drafts: Agent Memory (2606.06448 ✓), ThunderAgent (2602.13692 ✓), GoodServe (2605.16867 ✓), CONCUR (2601.22705 ✓), GAIATrace (2606.01725 ✓). Venue labels (ICLR’26 for Cost-of-Pass, ICML’26 for CONCUR, NSDI’26 for ServeGen) are *as-claimed* and not confirmed accepted.

